JAVA-

Q1. What is garbage collection in java?

When a program runs, it uses memory to store data (like variables and objects). Over time, some of that data is no longer needed—but it still sits in memory.

Garbage collection (GC) is the process of:

☞ finding that unused data (“garbage”)
☞ and freeing up the memory so it can be reused

Q2. What is dynamic binding in java?

Dynamic binding (also called **late binding** or **runtime polymorphism**) in Java means:

The method that gets executed is decided at **runtime**, not at compile time.

It mainly happens when:

- **method overriding** is used
- a **parent class reference** points to a **child class object**

✓ Example

```
class Animal {
   void sound() {
      System.out.println("Animal makes sound");
   }
}

class Dog extends Animal {
   @Override
   void sound() {
      System.out.println("Dog barks");
   }
}

public class Main {
   public static void main(String[] args) {

      Animal a = new Dog();   // parent reference, child object
      a.sound();              // Which sound() will run?
   }
}
```

◆ Output

Dog barks

Q3. Static Binding vs Dynamic Binding

Static Binding	Dynamic Binding
Compile-time	Runtime
Method call fixed early	Method call decided later
Used in method overloading	Used in method overriding
Faster	Slightly slower

Methods that use Static Binding -

These are resolved at compile time:

- static methods
- private methods
- final methods

Because they cannot be overridden.

Q4. What are Strings in java?

In Java, a **String** is an object used to store a sequence of characters (text).

Example:

String name = "Sudhanshu";

Here, "Sudhanshu" is a String.

Java provides the String class inside:

java.lang.String

Creating Strings-

1. Using String Literal

String s1 = "Hello";

Stored in the **String Constant Pool**.

2. Using `new` Keyword

```
String s2 = new String("Hello");
```

Creates a new object in heap memory.

Q5. String are immutable?

Strings are Immutable

Immutable means:

Once a String object is created, its value cannot be changed.

Example:

```
String s = "Hello";
s.concat(" World");

System.out.println(s);
```

Output

```
Hello
```

Because `concat()` creates a new String instead of modifying the old one.

Correct way:

```
s = s.concat(" World");
```

Q6. What is String Constant Pool (SCP)?

Java stores String literals in a special memory area called:

String Constant Pool

Example:

```
String a = "Java";
String b = "Java";
```

Both `a` and `b` point to the same object to save memory.

Q7. Common String Methods?

Method	Purpose
length()	Returns length
charAt()	Gets character
substring()	Extracts part
equals()	Compares content
equalsIgnoreCase()	Ignores case
toUpperCase()	Uppercase
toLowerCase()	Lowercase
trim()	Removes spaces
contains()	Checks text
replace()	Replaces characters

Q8. String vs StringBuilder vs StringBuffer

Feature	String	StringBuilder	StringBuffer
Mutable	✗ No	✓ Yes	✓ Yes
Thread Safe	✓ Yes	✗ No	✓ Yes
Performance	Slower for modifications	Fast	Moderate

Use:

- String → fixed text
- StringBuilder → frequent changes
- StringBuffer → thread-safe modifications

Q9. String Comparison?

1. Using = =

Compares memory addresses.

```
String a = "Java";
String b = "Java";

System.out.println(a == b);
```

Output:

true

2. Using `equals()`

Compares actual content.

```
String a = new String("Java");
String b = new String("Java");

System.out.println(a.equals(b));
```

Output:

True

```
String a = new String("Java");
String b = "Java";

System.out.println(a == b);
```

✓ Output

false

Q10. What are ways of creating thread in java?

In Java, there are **mainly 2 ways** to create a thread:

1. **By extending `Thread` class**
2. **By implementing `Runnable` interface**

1. Extending `Thread` Class

You create a class that extends `Thread` and override the `run()` method.

Example

```
class MyThread extends Thread {

    @Override
    public void run() {
        System.out.println("Thread is running...");
    }
}

public class Main {
    public static void main(String[] args) {
```

```
    MyThread t1 = new MyThread();

    t1.start();   // starts new thread
  }
}
```

🔍 Important

- **run()** contains the task
- **start()** creates a new thread internally and calls **run()**

If you call:

```
t1.run();
```

it behaves like a normal method call, not a separate thread.

✅ 2. Implementing `Runnable` Interface

This is the **preferred approach**.

Example

```
class MyRunnable implements Runnable {

  @Override
  public void run() {
    System.out.println("Runnable thread running...");
  }
}

public class Main {
  public static void main(String[] args) {

    MyRunnable obj = new MyRunnable();

    Thread t1 = new Thread(obj);

    t1.start();
  }
}
```

✅ Why Runnable is Preferred?

Because Java supports:

- single inheritance only

If you extend Thread, you cannot extend another class.

But with Runnable, your class can still extend another class.

3. Using Lambda Expression

Because Runnable is a functional interface.

```
public class Main {
   public static void main(String[] args) {

      Runnable r = () -> {
         System.out.println("Thread using lambda");
      };

      Thread t = new Thread(r);

      t.start();
   }
}
```

Q11. Difference Between Thread and Runnable

Thread Class	Runnable Interface
Extends Thread	Implements Runnable
Cannot extend another class	Can extend another class
Less flexible	More flexible
Tightly coupled	Better design

Q12. Thread Lifecycle and Common Thread Methods

--NEW → RUNNABLE → RUNNING → TERMINATED

Method	Purpose
start()	Starts thread
run()	Task logic
sleep()	Pause thread
join()	Wait for another thread
isAlive()	Checks if running

Q13. What are collenstions in java?

In Java, **Collections** are a framework used to store, manage, and manipulate groups of objects dynamically.

Main Interfaces in Collections -

Interface	Description
List	Ordered, duplicates allowed
Set	Unique elements
Queue	FIFO structure
Map	Key-value pairs

1. List Interface

Maintains insertion order and allows duplicates.

Common classes:

- ArrayList
- LinkedList
- Vector

Example

```
import java.util.*;

public class Main {

   public static void main(String[] args) {

      List<String> list = new ArrayList<>();

      list.add("Java");
      list.add("Python");
      list.add("Java");

      System.out.println(list);
   }
}
```

Output

```
[Java, Python, Java]
```

2. Set Interface

Stores unique elements only.

Common classes:

- HashSet
- LinkedHashSet
- TreeSet

Example

```
Set<Integer> set = new HashSet<>();

set.add(10);
set.add(20);
set.add(10);

System.out.println(set);
```

Output

```
[10, 20]
```

Duplicate 10 removed automatically.

3. Queue Interface

Follows FIFO (First In First Out).

Common classes:

- PriorityQueue
- LinkedList

Example

```
Queue<Integer> q = new LinkedList<>();

q.add(1);
q.add(2);

System.out.println(q.poll());
```

Output

```
1
```

4. Map Interface

Stores data as:

key -> value

Common classes:

- HashMap
- LinkedHashMap
- TreeMap

Example

```
Map<Integer, String> map = new HashMap<>();

map.put(1, "Java");
map.put(2, "Python");

System.out.println(map);
```

Output

```
{1=Java, 2=Python}
```

Q13. Common Collection Classes

Class	특징
ArrayList	Fast random access
LinkedList	Fast insertion/deletion
HashSet	Unique unordered data
TreeSet	Sorted unique data
HashMap	Key-value storage
PriorityQueue	Priority-based queue

Advantages of Collections

- Dynamic size
- Built-in algorithms
- Easy sorting/searching
- Reduces coding effort
- Generic support

✓ Generics in Collections

```
List<String> list = new ArrayList<>();
```

String means:

only String objects allowed

Provides:

- type safety
- compile-time checking

Important Utility Class

Collections class provides helper methods:

```
Collections.sort(list);
Collections.reverse(list);
Collections.shuffle(list);
```

Q14. What is bucketing in HashMap?

In Java, **bucketing in HashMap** refers to the way data is stored internally using **buckets**.

A bucket is basically:

a location in an internal array where key-value pairs are stored.

Q15. What is ConcurrentHashMap?

HashMap and ConcurrentHashMap are both used to store key-value pairs in Java, but the major difference is:

✓ HashMap is **not thread-safe**
✓ ConcurrentHashMap is **thread-safe**

Basic Difference

Feature	HashMap	ConcurrentHashMap
Thread-safe	✗ No	✓ Yes
Synchronization	None	Internal synchronization
Performance	Faster in single thread	Better for multi-threading
Null Keys	✓ One null key allowed	✗ Not allowed
Null Values	✓ Multiple allowed	✗ Not allowed
Iterator	Fail-fast	Fail-safe
Package	java.util	java.util.concurrent

1. HashMap

Used in:

- single-threaded applications
- when synchronization is not needed

Example

```
HashMap<Integer, String> map = new HashMap<>();

map.put(1, "Java");
map.put(2, "Python");
```

⚠ Problem with HashMap in Multithreading

If multiple threads modify HashMap simultaneously:

- data inconsistency may happen
- infinite loops may occur (older Java versions)
- corruption possible

2. ConcurrentHashMap

Designed for:

concurrent/multi-threaded environments

Example

```
ConcurrentHashMap<Integer, String> map =
    new ConcurrentHashMap<>();

map.put(1, "Java");
map.put(2, "Python");
```

? How ConcurrentHashMap Works

Older Java:

- segmented locking

Java 8+:

- bucket-level locking + CAS operations

This means:

multiple threads can work simultaneously on different buckets

So performance is much better than full synchronization.

🔍 Null Handling Difference

HashMap

map.put(null, "A");
map.put(1, null);

✅ Allowed

ConcurrentHashMap

map.put(null, "A");

❌ Throws:

NullPointerException

📌 Iterator Difference

HashMap → Fail-Fast

If map changes during iteration:

ConcurrentModificationException

ConcurrentHashMap → Fail-Safe

Allows modification during iteration safely.

✅ Example of Thread Safety

```
Map<Integer, String> map =
      new ConcurrentHashMap<>();

Runnable task = () -> {
   for(int i = 0; i < 1000; i++) {
      map.put(i, "Value");
   }
};

new Thread(task).start();
new Thread(task).start();
```

Safe for concurrent access.

----------**NEW BELOW**----------

Q1. What is Java?

ANS - Java is a high-level, object-oriented programming language that follows the WORA principle. Java code is compiled into bytecode which runs on JVM, making it platform independent.

Q2. Features of Java

ANS - **S O P M R M G**

- Simple
- Object Oriented
- Platform Independent
- Multithreaded
- Robust
- Memory Managed
- Garbage Collection

Q3. Why Java is Platform Independent?

Because of JVM.

```
Java Code
↓
Compiler
↓
Bytecode (.class)
↓
JVM
↓
Operating System
```

Different OS have different JVMs.

Q4. Difference between JDK, JRE and JVM

JDK	JRE	JVM
Development Kit	Runtime Environment	Executes Bytecode
Contains Compiler	Contains JVM	Converts Bytecode to Machine Code
Used by Developer	Used by User	Used by Both

Interview Question:

Can Java program run without JDK?

Answer:
Yes.

Can Java program run without JVM?

No.

Q5. Why Java is Object-Oriented?

Because it supports

- Class
- Object
- Inheritance
- Encapsulation
- Polymorphism
- Abstraction

Q6. why is java not called a pure object oriented language?

ANS - Java is not called a pure object-oriented language primarily because **it supports primitive data types (int, char, boolean, etc.) that are not objects**, violating the foundational "everything is an object" rule of pure Object-Oriented Programming (OOP). While Java thoroughly implements core OOP principles like encapsulation, inheritance, polymorphism, and abstraction, it maintains certain non-object features for performance and historical reasons.

Q7. Explain OOPS concepts.

ANS - 1. Encapsulation

Definition - Wrapping variables and methods into a single unit (class) and restricting direct access to data.

Data is hidden using

- private variables
- public getters/setters

Real-Life Example

ATM Machine - You don't know how money is processed internally.

You only - Insert card , Enter PIN , Withdraw money .

Internal implementation is hidden.

Java Example

```
class Employee {

    private String name;
    private int salary;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getSalary() {
        return salary;
    }

    public void setSalary(int salary) {

        if(salary > 0){
            this.salary = salary;
        }
    }
}

public class Main {

    public static void main(String[] args) {

        Employee emp = new Employee();

        emp.setName("John");
        emp.setSalary(50000);

        System.out.println(emp.getName());
        System.out.println(emp.getSalary());
    }
}
```

Output

```
John
50000
```

Why private?

Imagine - `salary = -5000;`

Should this happen? No. Setter validates data.

```
if(salary>0)
```

This is encapsulation.

Advantages

- Security
- Data hiding
- Easy maintenance
- Validation possible
- Loose coupling

Interview Question

Why use getter/setter?

Because variables are private.

Without setter, Anyone can modify

```
salary=-10000;
```

Getter/setter provide validation.

2. Abstraction

Definition - Showing only necessary details and hiding implementation.

Focus on - "What", instead of - "How"

Real-Life Example

Car, You only press

```
Start, Accelerate, Brake
```

You don't know how engine starts.

Java Example using Interface

```
interface Payment {

    void pay();
}

class CreditCard implements Payment {

    @Override
    public void pay() {
        System.out.println("Paid using Credit Card");
    }
}
```

```
class UPI implements Payment {

    @Override
    public void pay() {
        System.out.println("Paid using UPI");
    }
}

public class Main {

    public static void main(String[] args) {

        Payment payment = new UPI();
        payment.pay();
    }
}
```

Output

```
Paid using UPI
```

User only knows

```
pay()
```

Not implementation.

Abstract Class Example

```
abstract class Animal {

    abstract void sound();

    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {

    void sound() {
        System.out.println("Bark");
    }
}
```

Interface vs Abstract Class

Interface	Abstract Class
Multiple inheritance	Single inheritance
Only contract	Partial implementation
implements	extends
Variables are public static final	Can have instance variables

---

Advantages

- Security
- Flexibility
- Loose coupling
- Easier maintenance

3. Inheritance

Definition - One class acquires properties of another class.

Keyword - `extends`

Real-Life Example

Father → Son

Son inherits

- house
- car
- surname

Java Example

```
class Animal {

    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {

    void bark() {
        System.out.println("Barking");
    }
}
```

```
public class Main {

    public static void main(String[] args) {

        Dog dog = new Dog();

        dog.eat();
        dog.bark();
    }
}
```

Output

```
Eating
Barking
```

Types of Inheritance in Java

Single

```
A
↓

B
```

Multilevel

```
Animal

↓

Dog

↓

BabyDog
```

Hierarchical

```
      Animal
      /    \
    Dog    Cat
```

Multiple

Java doesn't support

A

B

↓

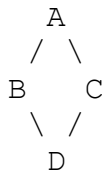
C

using classes.

Possible using interfaces.

Why Multiple Inheritance isn't supported?

Diamond Problem



Both B and C have same method.

Which one should D call?

Ambiguity. Hence Java avoids it.

4. Polymorphism

Definition - One object behaves in multiple forms.

Means - One interface, Multiple implementations.

Two types

- Compile-time
- Runtime

Compile-Time Polymorphism

Method Overloading - Same method, Different parameters.

Example

```
class Calculator {

    int add(int a,int b){
        return a+b;
    }

    int add(int a,int b,int c){
        return a+b+c;
    }
}
```

Output

```
add(2,3)
add(2,3,4)
```

Compiler decides.

Runtime Polymorphism → Method Overriding

```
class Animal{

    void sound(){
        System.out.println("Animal Sound");
    }
}

class Dog extends Animal{

    @Override
    void sound(){
        System.out.println("Bark");
    }
}

public class Main{

    public static void main(String[] args){

        Animal animal=new Dog();

        animal.sound();
    }
}
```

Output

```
Bark
```

JVM decides.

Why Runtime?

Because actual object

```
new Dog()
```

is known during runtime.

Advantages

- Loose coupling
- Extensible
- Reusable
- Easy maintenance

Overloading vs Overriding

Overloading	Overriding
Same class	Parent & Child
Same method	Same method
Different parameters	Same parameters
Compile time	Runtime
Static binding	Dynamic binding

Constructor

Special method called automatically.

```
class Student{

    Student(){

        System.out.println("Constructor Called");
    }
}
```

Output

```
Constructor Called
```

Types – Default, Parameterized, Copy Constructor (user-defined in Java)

---

this Keyword

Current object reference.

```
class Student{

    String name;

    Student(String name){

        this.name=name;
    }
}
```

---

super Keyword

Calls parent class constructor.

```
class Animal{

    Animal(){

        System.out.println("Animal");
    }
}

class Dog extends Animal{

    Dog(){

        super();

        System.out.println("Dog");
    }
}
```

Output

```
Animal

Dog
```

Association

Relationship between two independent objects.

Example

`Teacher → Student`

Teacher teaches Student.

Aggregation

Has-A relationship.

Weak ownership.

Example

`Department → Employee`

Employee can exist without Department.

Composition

Strong Has-A relationship.

Example

`House → Room`

Room cannot exist independently.

Object Class

Every class extends

`Object`

Important methods

`toString(), equals(), hashCode(), clone(), wait(), notify(), getClass()`

Object Creation

```
Student s = new Student();
```

Steps

1. Memory allocated in Heap
2. Constructor called
3. Object reference stored in Stack

Heap vs Stack

Stack	Heap
Local variables	Objects
Method calls	Instance variables
Fast	Larger memory
Automatically cleared	GC cleans

Access Modifiers

Modifier	Same Class	Same Package	Subclass	Other Package
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No (unless through inheritance)
public	Yes	Yes	Yes	Yes

Interview Questions (Most Asked)

Q1. Why Java is called an Object-Oriented Language?

Because it organizes programs around objects and supports encapsulation, abstraction, inheritance, and polymorphism.

Q2. Can we achieve abstraction without an interface?

Yes. By using an abstract class.

Q3. Can we instantiate an abstract class?

No.

Q4. Can constructors be inherited?

No.

Q5. Can private methods be overridden?

No, because they are not visible to subclasses.

Q6. Why is method overloading called compile-time polymorphism?

The compiler determines which overloaded method to invoke based on the method signature during compilation.

Q7. Why is method overriding called runtime polymorphism?

The JVM chooses the method implementation at runtime based on the actual object type, enabling dynamic method dispatch.

Q8. What is the difference between IS-A and HAS-A relationships?

- **IS-A** → Inheritance (`Dog IS-A Animal`)
- **HAS-A** → Composition/Aggregation (`Car HAS-A Engine`)

Q9. Can constructor be final?

No.

Q10. Can constructor be static?

No.

Q11. Can constructor return value?

No.

Interview Tip (3 Years Experience)

Don't just define OOP concepts—relate them to your project. For example:

- **Encapsulation:** "In my Spring Boot application, I keep entity fields private and expose them through getters/setters with validation."
- **Abstraction:** "I use service interfaces like `UserService` and hide the implementation in `UserServiceImpl`."
- **Inheritance:** "I extend a base exception or base entity class to reuse common functionality."
- **Polymorphism:** "In Spring, dependency injection injects different implementations of an interface, and method overriding allows the appropriate implementation to be called at runtime."

Q8 . What is static keyword?

ANS - Belongs to class. Not object.

Example

```
class Student{

static String college="ABC";

}
```

Interview Question -

Can static method access non-static variable?

No.

Q9. final Keyword

ANS – Variable → Cannot change value.

Method → Cannot override.

Class → Cannot inherit

Q10. finally vs finalize vs final

ANS - 1. final Keyword

Definition - `final` is a **keyword** used to restrict modification.

It can be applied to:

- Variable
- Method
- Class

Think of it as **"cannot be changed."**

A. final Variable

A final variable can be assigned **only once**.

```
public class Demo {

    public static void main(String[] args) {

        final int age = 25;

        // age = 30; // Compile-time Error

        System.out.println(age);
    }
}
```

Output

```
25
```

Why use it?

Suppose GST rate should never change.

```
final double GST = 18.0;
```

No one can accidentally modify it.

Blank Final Variable

Can be initialized inside the constructor.

```
class Employee {

    final int id;

    Employee(int id) {
        this.id = id;
    }

    void display() {
        System.out.println(id);
    }
}
```

Each object can have a different ID, but once assigned, it cannot change.

---

final Reference Variable

A reference cannot point to another object, but the object's state can still change.

```
class Student {

    String name;
}

public class Demo {

    public static void main(String[] args) {

        final Student s = new Student();

        s.name = "John";      // Allowed

        // s = new Student(); // Compile-time Error

        System.out.println(s.name);
    }
}
```

Interview Trap

Can we modify the object if the reference is final?

Answer: Yes. The object can be modified; only the reference cannot be reassigned.

B. final Method

A final method cannot be overridden.

```
class Animal {

    final void sound() {
        System.out.println("Animal Sound");
    }
}

class Dog extends Animal {

    // void sound() {} // Compile-time Error
}
```

Why? → To prevent subclasses from changing critical behavior.

Example:

```
calculateSalary()
authenticate()
```

C. final Class → A final class cannot be inherited.

```
final class Car {

}
class BMW extends Car {

}
```

Compile-time Error.

Why make a class final?

- Security
- Immutability
- Prevent misuse

Example: - String is a final class.

Why?

Imagine if someone could extend `String` and change the behavior of `equals()` or `hashCode()`. It would break Java's core functionality.

Advantages of final

- Prevents accidental modification
- Improves security
- Enables immutable objects
- Can help the JVM with certain optimizations (implementation-dependent)

---

2. finally Block

Definition - `finally` is a block used with exception handling.

It executes **whether an exception occurs or not**. Its main purpose is **resource cleanup**.

Example 1

```
public class Demo {

    public static void main(String[] args) {

        try {

            int result = 10 / 2;

            System.out.println(result);

        } finally {

            System.out.println("Finally Executed");
        }
    }
}
```

Output

```
5
Finally Executed
```

Example 2

```
public class Demo {

    public static void main(String[] args) {

        try {

            int result = 10 / 0;

        } catch (Exception e) {

            System.out.println("Exception");

        } finally {

            System.out.println("Finally");
        }
    }
}
```

Output

```
Exception
Finally
```

Why use finally?

Imagine:

```
File file = open();
```

Even if an exception occurs,

- close file
- close database connection
- close network connection

must happen.

That's why we use:

```
finally {

    connection.close();
}
```

Can finally be skipped?

Normally, **no**. However, there are exceptions, for example:

```
System.exit(0);
try {

    System.exit(0);

} finally {

    System.out.println("Won't Execute");
}
```

Output

`Program Ends,` Because the JVM terminates before executing the finally block.

Modern Java Note

Today, for resources like files or database connections, **try-with-resources** is usually preferred over manually closing them in `finally`.

```
try (BufferedReader reader = new BufferedReader(new
FileReader("data.txt"))) {
    System.out.println(reader.readLine());
}
```

The resource is closed automatically.

---

3. finalize() Method

Definition - `finalize()` is a method that was intended to be called by the Garbage Collector before reclaiming an object.

```
protected void finalize() throws Throwable {

}
```

Example

```
class Demo {

    @Override
    protected void finalize() throws Throwable {

        System.out.println("Finalize Called");
    }

    public static void main(String[] args) {

        Demo d = new Demo();

        d = null;

        System.gc();
    }
}
```

Possible Output

```
Finalize Called
```

Why was finalize() used?

Earlier, developers used it to release unmanaged resources before an object was collected.

Examples:

- Close files
- Release native resources

Why is finalize() not recommended now?

Because:

- The Garbage Collector decides **if and when** it runs.
- It may **never execute**.
- It causes performance issues.
- It is unpredictable.

Java Interview Fact

`finalize()` was **deprecated in Java 9** and later **removed/deprecated for removal** in newer Java versions. Modern Java recommends alternatives such as **try-with-resources** and the `Cleaner` API when appropriate.

Frequently Asked Interview Questions

Q1. Can a final class have constructors?

Yes. A `final` class can have constructors because objects still need to be created.

Q2. Can a final method be overloaded?

Yes.

```
final void print() {

}

void print(int a) {

}
```

Overloading is allowed because it creates a different method signature.

Q3. Can a constructor be final?

No. Constructors are not inherited, so preventing overriding makes no sense.

Q4. Can local variables be final? → Yes. **`final int x = 10;`**

Q5. Can finally exist without catch?

Yes.

```
try {

    System.out.println("Hello");

} finally {

    System.out.println("Done");
}
```

This is valid Java.

Q6. Can try exist without catch and finally?

Only if it is a **try-with-resources** statement.

```
try (Scanner sc = new Scanner(System.in)) {
    System.out.println(sc.nextLine());
}
```

A plain `try { ... }` without `catch`, `finally`, or resources is invalid.

Q7. Does finally execute after a return statement?

Yes.

```
public class Demo {

    static int test() {

        try {

            return 10;

        } finally {

            System.out.println("Finally");
        }
    }

    public static void main(String[] args) {

        System.out.println(test());
    }
}
```

Output

```
Finally
10
```

The `finally` block runs before the method actually returns.

Q8. Can finally change the return value?

Yes, but it is strongly discouraged.

```
static int test() {

    try {
        return 10;
    } finally {
        return 20;
    }
}
```

Output

```
20
```

The `finally` return overrides the `try` return, making code confusing and error-prone.

A strong overall interview answer would be:

"`final` is a keyword used to prevent modification of variables, methods, or classes. `finally` is an exception-handling block that is primarily used for cleanup activities such as closing database connections or files, though in modern Java we usually prefer try-with-resources. `finalize()` was a method intended to be invoked by the garbage collector before reclaiming an object, but it is deprecated because its execution is unpredictable and it can negatively impact performance. Modern Java applications should avoid using `finalize()`."

Q11. Pass by Value or Reference?

Java is always - **Pass by Value**

Even object references are passed by value (the value being copied is the reference).

---------- SOME CODING QUESTIONS BELOW ----------

1. Reverse String

Ans - **Approach 1 (Most Preferred)**

```
public class ReverseString {

    public static void main(String[] args) {

        String s = "Java";
        String reverse = "";

        for (int i = s.length() - 1; i >= 0; i--) {
            reverse += s.charAt(i);
        }

        System.out.println(reverse);
    }
}
```

Output → `avaJ` **Time Complexity →** $O(n^2)$, Because String is immutable.

Optimized Approach (Interview Preferred)

```
public class ReverseString {

    public static void main(String[] args) {

        String s = "Java";

        StringBuilder sb = new StringBuilder();

        for (int i = s.length() - 1; i >= 0; i--) {
            sb.append(s.charAt(i));
        }

        System.out.println(sb);
    }
}
```

Time → O(n)

Interview POV

Possible follow-ups:

- Reverse without using `StringBuilder.reverse()`.
- Reverse words instead of characters.
- Reverse recursively.

---

2. Palindrome

Input → `madam`, Output → `Palindrome`

```
public class Palindrome {

    public static void main(String[] args) {

        String s = "madam";
        String reverse = "";

        for (int i = s.length() - 1; i >= 0; i--) {
            reverse += s.charAt(i);
        }

        if (s.equals(reverse)) {
            System.out.println("Palindrome");
        } else {
            System.out.println("Not Palindrome");
        }
    }
}
```

Better Approach -

Use two pointers instead of creating a new string.

```
public class Palindrome {

    public static void main(String[] args) {

        String s = "madam";

        int left = 0;
        int right = s.length() - 1;

        boolean palindrome = true;

        while (left < right) {

            if (s.charAt(left) != s.charAt(right)) {
                palindrome = false;
                break;
            }

            left++;
            right--;
        }

        System.out.println(palindrome ? "Palindrome" : "Not
Palindrome");
    }
}
```

Time → O(n), Space → O(1).

---

3. Count Vowels → Input → Education, Output → Vowels = 5

```
public class CountVowels {

    public static void main(String[] args) {

        String s = "Education";

        int count = 0;

        s = s.toLowerCase();

        for (char ch : s.toCharArray()) {

            if (ch == 'a' || ch == 'e' || ch == 'i'
                    || ch == 'o' || ch == 'u') {

                count++;
            }
        }

        System.out.println(count);
    }
}
```

Time → O(n)

---

4. Remove Duplicate Characters

Input → Programming, Output → Progamin

Using LinkedHashSet (Preserves Order)

```
import java.util.LinkedHashSet;

public class RemoveDuplicate {

    public static void main(String[] args) {

        String s = "Programming";

        LinkedHashSet<Character> set = new LinkedHashSet<>();

        for (char ch : s.toCharArray()) {
            set.add(ch);
        }

        for (char ch : set) {
            System.out.print(ch);
        }
    }
}
```

Output → Progamin, Time → O(n)

Without Collections (Frequently Asked)

```
public class RemoveDuplicate {

    public static void main(String[] args) {

        String s = "Programming";
        String result = "";

        for (char ch : s.toCharArray()) {

            if (result.indexOf(ch) == -1) {
                result += ch;
            }
        }

        System.out.println(result);
    }
}
```

5. Second Largest Number

Array → `10 25 45 80 60`

Output → `60`

```
public class SecondLargest {

    public static void main(String[] args) {

        int arr[] = {10, 25, 45, 80, 60};

        int largest = Integer.MIN_VALUE;
        int second = Integer.MIN_VALUE;

        for (int num : arr) {

            if (num > largest) {

                second = largest;
                largest = num;

            } else if (num > second && num != largest) {

                second = num;
            }
        }

        System.out.println(second);
    }
}
```

Time → `O(n)`, Space → `O(1)`

---

6. Factorial

Input → `5`, Output → `120`

```
public class Factorial {

    public static void main(String[] args) {

        int n = 5;

        int fact = 1;

        for (int i = 1; i <= n; i++) {

            fact *= i;
        }

        System.out.println(fact);
    }
}
```

Time → `O(n)`

7. Prime Number

Input → `17`, Output → `Prime`

```
public class Prime {

    public static void main(String[] args) {

        int n = 17;

        boolean prime = true;

        if (n <= 1) {
            prime = false;
        } else {

            for (int i = 2; i <= Math.sqrt(n); i++) {

                if (n % i == 0) {

                    prime = false;
                    break;
                }
            }
        }

        System.out.println(prime ? "Prime" : "Not Prime");
    }
}
```

Time → `O(√n)`

---

8. Fibonacci Series →

Output → `0 1 1 2 3 5 8 13`

```
public class Fibonacci {

    public static void main(String[] args) {

        int n = 8;

        int a = 0;
        int b = 1;

        System.out.print(a + " " + b + " ");

        for (int i = 2; i < n; i++) {

            int c = a + b;

            System.out.print(c + " ");

            a = b;
            b = c;
        }
    }
}
```

Time → `O(n)`

9. Anagram

Input → `listen, silent`

Output → `Anagram`

```
import java.util.Arrays;

public class Anagram {

    public static void main(String[] args) {

        String s1 = "listen";
        String s2 = "silent";

        char a[] = s1.toCharArray();
        char b[] = s2.toCharArray();

        Arrays.sort(a);
        Arrays.sort(b);

        System.out.println(Arrays.equals(a, b)
                ? "Anagram"
                : "Not Anagram");
    }
}
```

Time → `O(n log n)`

Better (Using Frequency Count)

Use an `int[26]` array for lowercase English letters to achieve **O(n)** time.

---

10. Frequency of Characters

Input → `Programming`

Output → `P -> 1, r -> 2, o -> 1, g -> 2, a -> 1, m -> 2, i -> 1, n -> 1`

```
import java.util.HashMap;

public class Frequency {

    public static void main(String[] args) {

        String s = "Programming";

        HashMap<Character, Integer> map = new HashMap<>();

        for (char ch : s.toCharArray()) {

            map.put(ch, map.getOrDefault(ch, 0) + 1);
        }

        System.out.println(map);
    }
}
```

Time → `O(n)`

---------- NEW QUESTIONS ----------

Q1. Why main() is static?

JVM calls it without creating object.

Q2. Can we override private method?

No.

Q3. Can we override static method?

No (method hiding occurs instead).

Q4. Can we overload static method?

Yes.

Q5. Difference between Array and ArrayList?

Array	ArrayList
Fixed size	Dynamic size
Primitive + Objects	Objects only (autoboxing for primitives)
Faster	More flexible

Q6. Why Java doesn't support pointers?

Security, Memory safety, Simplicity

Q7. What is Lambda Expression?

Answer → A Lambda Expression is an anonymous function introduced in Java 8 that allows us to write concise code by implementing Functional Interfaces.

Instead of creating anonymous inner classes, we write:

```
(parameters) -> expression
```

or

```
(parameters) -> {
    body;
}
```

Before Java 8

```
Runnable r = new Runnable() {
    @Override
    public void run() {
        System.out.println("Running");
    }
};
```

Java 8

```
Runnable r = () -> System.out.println("Running");
```

Much shorter.

Syntax

```
() -> {}
```

No parameter

```
x -> x*x
```

One parameter

```
(a,b) -> a+b
```

Multiple parameters

```
(name) -> {
    System.out.println(name);
}
```

Multiple statements

Why Lambda?

Without lambda

```
Collections.sort(list, new Comparator<Integer>() {
    public int compare(Integer a,Integer b){
        return a-b;
    }
});
```

With Lambda

```
Collections.sort(list,(a,b)->a-b);
```

Cleaner.

Internal Working

When compiler sees

```
x -> x+1
```

It converts it into an implementation of Functional Interface using **invokedynamic** (JVM optimization).

Unlike anonymous class, Lambda does NOT generate an extra .class file.

Advantages → Less code, Readable, Functional Programming, Easy Collection Processing, Parallel Processing

Q9. Method Reference

Instead of -

```
x->System.out.println(x)
```

Write -

```
System.out::println
```

Example

```
List<String> list=List.of("A","B","C");

list.forEach(System.out::println);
```

Types

Static Method

```
Math::abs
```

Instance Method

```
System.out::println
```

Constructor

```
ArrayList::new
```

Difference between Lambda and Method Reference?

Lambda →

```
x->System.out.println(x)
```

Method Reference →

```
System.out::println
```

Method Reference is shorter but only works when the lambda body directly calls an existing method.

----------MULTITHREADING AND CONCURRENCY----------

1. Process vs Thread

Process

A process is an independent program running in memory.

Example → `Chrome, VS Code, Spotify`

```
Each is a Process.
```

Each process has

- Separate Memory
- Separate Heap
- Separate Stack

Thread → A thread is the smallest unit of execution inside a process.

Example → Chrome

```
Chrome Process

|
|----UI Thread
|
|----Download Thread
|
|----Render Thread
|
|----Network Thread
```

All threads

- Share Heap Memory
- Share Objects

But

Each thread has its own

- Stack
- Program Counter

Difference between Process and Thread?

Process	Thread
Heavyweight	Lightweight
Separate memory	Shared memory
Slow creation	Fast creation
IPC needed	Easy communication
Crash isolated	One thread may affect process

2. What is Multithreading?

→ Running multiple threads simultaneously.

Example

Amazon App

```
Loading products

+
Downloading Images

+
Showing UI

+
Fetching Recommendations
```

All happen simultaneously.

Advantages → ✓ Faster, ✓ Better CPU utilization, ✓ Responsive application

---

3. Ways to Create Thread

Method 1 → Extend Thread

```
class MyThread extends Thread {

    public void run() {
        System.out.println("Thread Running");
    }

    public static void main(String[] args) {

        MyThread t = new MyThread();
        t.start();
    }
}
```

Output

```
Thread Running
```

Method 2 → Implement Runnable

```
class MyRunnable implements Runnable {

    public void run() {
        System.out.println("Runnable Running");
    }

    public static void main(String[] args) {

        Thread t = new Thread(new MyRunnable());

        t.start();
    }
}
```

Why call `start()` instead of `run()`?

```
start()
↓
Creates new thread
↓
Calls run()
```

If you call → `run();` → No new thread is created.

It behaves like a normal method call.

4. Runnable vs Thread

Runnable	Thread
Interface	Class
Preferred	Not preferred
Supports multiple inheritance	No
Better design	Tight coupling

Interview Answer

Runnable is preferred because Java supports only single inheritance. It separates the task from the thread and allows better reuse.

5. Thread Life Cycle →

```
NEW

↓

RUNNABLE

↓

RUNNING

↓

WAITING

↓

TIMED_WAITING

↓

TERMINATED
```

Example

```
new Thread()

↓

start()

↓

run()

↓
```

```
sleep()

↓

finish
```

---

6. sleep() →

```
Thread.sleep(2000);
```

Current thread sleeps for 2 seconds.

Example

```
System.out.println("Start");

Thread.sleep(2000);

System.out.println("End");
```

Output

```
Start

(wait)

End
```

Does sleep release lock?

Answer → ✕ No.

---

7. join() →

Suppose, Main thread waits for child thread.

```
Thread t=new Thread(...);

t.start();

t.join();

System.out.println("Done");
```

Output

```
Child finishes

↓

Done
```

What does join() do?

Answer → Current thread waits until another thread finishes.

---

8. Yield() →

```
Thread.yield();
```

Suggestion to scheduler "I can wait." Not guaranteed.

---

9. Synchronization →

Most important topic.

Suppose → `Account Balance =1000`

Thread A → Withdraw 500

Thread B → Withdraw 500

Without synchronization

```
Both read 1000

↓

Both update

↓

Wrong balance
```

This is called Race Condition.

Example

```
class Counter {

    int count = 0;

    synchronized void increment() {
        count++;
    }
}
```

Without synchronized

```
count++

↓

Read

↓

Increment

↓

Write
```

Three operations. Two threads may interfere.

Why synchronize?

To avoid race condition and ensure thread safety.

---

10. Race Condition →

Example → count++; Looks simple.

Actually

```
Read

Increment

Write
```

Multiple threads

↓

Wrong answer

Example

```
Counter c = new Counter();

Thread t1 = new Thread(() -> {
    for(int i=0;i<1000;i++)
        c.increment();
});

Thread t2 = new Thread(() -> {
    for(int i=0;i<1000;i++)
        c.increment();
});

t1.start();
t2.start();

t1.join();
t2.join();

System.out.println(c.count);
```

Expected

```
2000
```

Without synchronization

Maybe

```
1948

1990

1973
```

Random.

---

11. synchronized Block →

```
synchronized(this){

}
```

Example

```
void increment() {

    synchronized(this){
        count++;
    }

}
```

Better when only part of method needs locking.

---

12. Static Synchronization

```
public static synchronized void print() {

}
```

Locks Class object. Not instance.

---

13. Deadlock →

Most favorite interview question.

Example

Thread 1

```
Lock A

↓

Lock B
```

Thread 2

```
Lock B

↓

Lock A
```

Now Both wait forever. Deadlock.

Example

```
Thread1

synchronized(A){

    synchronized(B){

    }

}
```

Thread2

```
synchronized(B){

    synchronized(A){

    }

}
```

Avoid, Always acquire locks in same order.

---

14. volatile →

Without volatile Thread may cache variable.

Example

```
boolean flag = false;
```

Thread1 → `flag=true;`

Thread2 → May still see `false`

Use → volatile boolean flag;

Now every thread reads latest value.

Does volatile provide thread safety? → No.

It provides Visibility only.

---

15. AtomicInteger →

Instead of

```
count++;
```

Use

```
AtomicInteger count=new AtomicInteger();
```

Increment

```
count.incrementAndGet();
```

No synchronization required.

Example

```
AtomicInteger count=new AtomicInteger();

count.incrementAndGet();

System.out.println(count.get());
```

AtomicInteger vs synchronized?

AtomicInteger uses CAS (Compare-And-Swap), which is often faster for simple atomic operations because it avoids blocking. `synchronized` can protect larger critical sections and multiple operations together.

---

16. Executor Framework →

Instead of → `1000 Threads`

Use → `Thread Pool`

Example

```
ExecutorService executor = Executors.newFixedThreadPool(3);

executor.submit(() ->
    System.out.println("Hello"));

executor.shutdown();
```

Advantages

- Reuse Threads
- Better Performance
- Less Memory
- Better Resource Management

Why ExecutorService?

Thread creation is expensive.

Executor reuses threads.

---

17. Callable →

Runnable --. `No Return`

`No Exception`

Callable → `Returns value`

`Throws Exception`

Example

```
Callable<Integer> task = () -> 100;

ExecutorService executor =
Executors.newSingleThreadExecutor();

Future<Integer> future =
executor.submit(task);

System.out.println(future.get());

executor.shutdown();
```

Output

```
100
```

---

18. Future →

Represents result of asynchronous task.

```
Future<String> future=
executor.submit(() -> "Java");
```

Later

```
future.get();
```

Gets result.

---

19. ConcurrentHashMap →

Instead of → HashMap

Use → ConcurrentHashMap

Supports concurrent access safely.

HashMap vs ConcurrentHashMap

HashMap	ConcurrentHashMap
Not Thread Safe	Thread Safe
Faster (single thread)	Concurrent access
May throw issues under concurrent modification	Designed for concurrency

---

20. CountDownLatch →

Suppose Need to wait for → `Database, API, File` - **Then proceed.**

```
CountDownLatch latch = new CountDownLatch(3);
```

Workers → `latch.countDown();`

Main → `latch.await();`

Example 1: Basic Program →

```
import java.util.concurrent.CountDownLatch;

public class CountDownLatchExample {

    public static void main(String[] args) throws InterruptedException {

        CountDownLatch latch = new CountDownLatch(3);

        Runnable task = () -> {

            System.out.println(Thread.currentThread().getName() + " Started");

            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }

            System.out.println(Thread.currentThread().getName() + " Finished");

            latch.countDown();
        };

        new Thread(task, "Service-1").start();
        new Thread(task, "Service-2").start();
        new Thread(task, "Service-3").start();

        System.out.println("Main Thread Waiting...");

        latch.await();

        System.out.println("All Services Completed");
    }
}
```

Output

```
Service-1 Started
Service-2 Started
Service-3 Started

Main Thread Waiting...

Service-2 Finished
Service-1 Finished
Service-3 Finished

All Services Completed
```

Notice:

The main thread **waits** until all three services finish.

How It Works Internally

Initially → `Count = 3`

After Service 1 finishes → `Count = 2`

After Service 2 finishes → `Count = 1`

After Service 3 finishes → `Count = 0`

Now

```
await()
↓
Unblocked
↓
Main Thread Continues
```

Important Methods

1. Constructor

```
CountDownLatch latch = new CountDownLatch(3);
```

Creates a latch with count = 3.

2. countDown()

```
latch.countDown();
```

Reduces count by **1**.

Example

```
3 → 2 → 1 → 0
```

Once it reaches zero,

Waiting threads are released.

3. await()

`latch.await();` → `Current thread waits. If count becomes zero,Execution resumes.`

4. getCount()

```
System.out.println(latch.getCount());
```

Output

```
3

2

1

0
```

Useful for debugging.

---

21. Semaphore →

Suppose Parking Only 3 cars allowed.

```
Semaphore s = new Semaphore(3);
```

Acquire → `s.acquire();`

Release → `s.release();`

----------EXCEPTION HANDLING BELOW----------

Q. What is an Exception?

An **Exception** is an event that interrupts the normal flow of a program.

Example

```
int a = 10;
int b = 0;

System.out.println(a / b);
```

Output

```
ArithmeticException: / by zero
```

Program crashes because an exception occurred.

Q. Why Exception Handling?

Without exception handling

```
public class Demo {

    public static void main(String[] args) {

        System.out.println("Start");

        int result = 10 / 0;

        System.out.println(result);

        System.out.println("End");
    }
}
```

Output

```
Start

Exception in thread "main"
ArithmeticException

Program terminated.
```

With exception handling

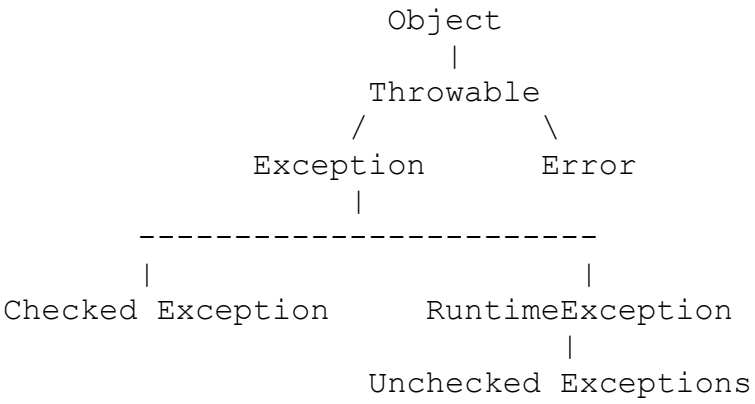
```
public class Demo {

    public static void main(String[] args) {

        System.out.println("Start");

        try {

            int result = 10 / 0;

            System.out.println(result);

        } catch (ArithmeticException e) {

            System.out.println("Cannot divide by zero");

        }

        System.out.println("End");
    }
}
```

Output

```
Start
Cannot divide by zero
End
```

Program continues.

Exception Hierarchy



Throwable

Root class.

Two children

- Exception
- Error

Error → Serious problem.

Examples

`OutOfMemoryError`

`StackOverflowError`

`VirtualMachineError`

Cannot usually recover.

Interview Answer

Errors are caused by JVM and generally should not be handled.

Exception → Recoverable problems.

Example

`IOException`

`SQLException`

`FileNotFoundException`

Types of Exceptions

1. Checked Exception

Checked at compile time.

Examples → `IOException, SQLException, ClassNotFoundException, FileNotFoundException`

Example

```
FileReader reader = new FileReader("abc.txt");
```

Compiler says

```
Unhandled exception FileNotFoundException
```

Must

- try-catch
  OR
- throws

---

2. Unchecked Exception

Occurs at runtime.

Examples → `ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, NumberFormatException`

Example

```
String s = null;

System.out.println(s.length());
```

Output

```
NullPointerException
```

Difference between Checked and Unchecked Exception?

Checked	Unchecked
Compile time	Runtime
Must handle	Optional
Extends Exception	Extends RuntimeException

---

try Block

Contains risky code.

```
try {
    int result = 10 / 0;
}
```

---

catch Block

Handles exception.

```
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println(e.getMessage());
}
```

Output

```
/ by zero
```

---

finally Block

Always executes.

Used for cleanup.

```
try {
    System.out.println("Try");
} catch(Exception e){
    System.out.println("Catch");
} finally {
    System.out.println("Finally");
}
```

Output

```
Try
Finally
```

Even if exception occurs

```
Try
Catch
Finally
```

Even if return statement exists

```
public static int test(){

    try{

        return 10;

    } finally{

        System.out.println("Finally");

    }

}
```

Output

```
Finally
10
```

Interview favorite.

Can finally not execute?

Yes, When `System.exit(), JVM crash, Power failure`

---

throw Keyword

Used to throw exception manually.

```
public class Demo {

    public static void main(String[] args) {

        throw new ArithmeticException("Age invalid");

    }
}
```

Output

```
ArithmeticException
Age invalid
```

---

throws Keyword

Declares exception.

```
public void readFile() throws IOException{

}
```

Meaning "I am not handling it. Caller should handle."

Difference between throw and throws

throw	throws
Used inside method	Used in method signature
Throws one exception	Can declare multiple
Object required	Class name required

Multiple Catch

```
try {

    int arr[] = {1,2};

    System.out.println(arr[5]);

} catch(ArrayIndexOutOfBoundsException e){

    System.out.println("Invalid index");

} catch(Exception e){

    System.out.println("General Exception");

}
```

Specific catch first.

Wrong

```
catch(Exception e){

}

catch(ArithmeticException e){

}
```

Compiler error. Because parent cannot come before child.

Multi Catch (Java 7)

```
try{
    String s = null;
    System.out.println(s.length());
}catch(NullPointerException | ArithmeticException e){
    System.out.println("Handled");
}
```

Nested try

```
try{
    try{
        int x = 10 / 0;
    }catch(Exception e){
        System.out.println("Inner");
    }
}catch(Exception e){
    System.out.println("Outer");
}
```

Output → `Inner`

---

try-with-resources (Java 7)

Automatically closes resources.

Without

```
BufferedReader br = new BufferedReader(new FileReader("a.txt"));
try{
}finally{
    br.close();
}
```

With

```
try(BufferedReader br = new BufferedReader(new FileReader("a.txt"))){
    System.out.println(br.readLine());
}
```

No need to close.Works only with `AutoCloseable`

Custom Exception

Example

```
class InvalidAgeException extends Exception{

    public InvalidAgeException(String msg){

        super(msg);

    }

}
```

Use

```
public class Demo {

    static void checkAge(int age)
            throws InvalidAgeException{

        if(age < 18){

            throw new InvalidAgeException("Not eligible");

        }

    }

    public static void main(String args[]){

        try{

            checkAge(16);

        }catch(Exception e){

            System.out.println(e.getMessage());

        }

    }

}
```

Output → `Not eligible`

Common Runtime Exceptions

ArithmeticException → 10/0

NullPointerException

```
String s = null;

s.length();
```

NumberFormatException → Integer.parseInt("ABC");

ArrayIndexOutOfBoundsException → arr[10];

ClassCastException

```
Object obj = "Java";

Integer x = (Integer)obj;
```

IllegalArgumentException → Thread.sleep(-10);

Exception Propagation

```
class Demo {

    static void m1(){

        m2();

    }

    static void m2(){

        m3();

    }

    static void m3(){

        int x = 10/0;

    }

}
```

Exception propagates

```
m3
↓
m2
↓
m1
↓
main
```

If nobody handles

JVM handles.

Important Exception Methods

```
try{

}catch(Exception e){

    e.printStackTrace();

    e.getMessage();

    e.toString();

}
```

Example

```
printStackTrace()

Full stack trace
getMessage()

/ by zero
toString()

java.lang.ArithmeticException: / by zero
```

Best Practices

✅ Catch specific exception

```
catch(IOException e)
```

Better than

```
catch(Exception e)
```

✅ Never swallow exception

Bad

```
catch(Exception e){

}
```

Good

```
catch(Exception e){

    e.printStackTrace();

}
```

✅ Use Custom Exception

Business validations

`InvalidEmployeeException`

`InsufficientBalanceException`

`OrderNotFoundException`

✅ Use try-with-resources

Avoid resource leaks.

Real Project Example

Spring Boot REST API

```
@GetMapping("/{id}")
public Employee getEmployee(@PathVariable int id){
    return repository.findById(id)
            .orElseThrow(() ->
             new EmployeeNotFoundException("Employee not found"));
}
```

Global Exception Handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(EmployeeNotFoundException.class)
    public ResponseEntity<String> handle(EmployeeNotFoundException ex){
        return ResponseEntity
                .status(HttpStatus.NOT_FOUND)
                .body(ex.getMessage());
    }
}
```

This is a common production pattern.

Frequently Asked Interview Questions

1. What is Exception Handling?

A mechanism to handle runtime errors and maintain the normal flow of the program.

---

2. Difference between Error and Exception?

Error	Exception
Serious JVM problem	Recoverable
Not handled	Can be handled

---

3. Difference between Checked and Unchecked Exception?

Checked exceptions are verified at compile time and must be handled or declared, while unchecked exceptions occur at runtime and are not required to be handled.

---

4. Difference between throw and throws?

- `throw` is used inside a method to explicitly throw an exception object.
- `throws` is used in the method signature to declare that a method may pass exceptions to its caller.

---

5. Can we have `try` without `catch`?

Yes, if it is followed by `finally`.

```
try {
    System.out.println("Work");
} finally {
    System.out.println("Cleanup");
}
```

---

6. Can we have `catch` without `try`?

No.

---

7. Can we have multiple `catch` blocks?

Yes, but always place the most specific exceptions before more general ones.

---

8. Can `finally` block be skipped?

Normally no. It is skipped only in situations like `System.exit()`, JVM termination, or abrupt process failure.

---

9. Can we throw our own exception?

Yes, by creating a custom exception class that extends `Exception` (checked) or `RuntimeException` (unchecked).

---

10. What is exception propagation?

If a method does not handle an exception, it is passed up the call stack until a handler is found or the JVM terminates the program.